ТОО ”Колледж Хекслет”

Лабораторная работа №9

Интеграция REST API с клиентским приложением

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_2. Интеграция REST API с клиентским приложением.

Цель: Подключить браузерный frontend к разработанному Product REST API и реализовать из интерфейса получение, создание, изменение и удаление товаров.

Условие: RO4_2/lesson9/ЛР-9.pdf

Требования задания

• открываться через Express;

• использовать каталог public;

• получать товары через REST API;

• не хранить начальный список товаров непосредственно в HTML;

• выполнять GET через fetch();

• отправлять POST из HTML-формы;

• отправлять JSON с Content-Type: application/json;

• выполнять PATCH выбранного товара;

• выполнять DELETE выбранного товара;

• связывать DOM-элемент с ресурсом через id;

• проверять response.ok;

• показывать ошибку пользователю;

• обновлять DOM после изменения серверных данных;

• использовать отдельные api.js, ui.js и app.js.

Ход выполнения

Express отдаёт public. api.js содержит fetch-запросы и проверку response.ok; ui.js безопасно создаёт DOM через textContent; app.js связывает форму и кнопки с CRUD. После изменения выполняется повторный GET. DELETE требует подтверждения, PATCH изменяет цену выбранного id. Ошибки API выводятся на странице.

Проект: RO4_2/lesson9

Запуск из каталога проекта: npm install, затем npm start. Адрес сервера: http://localhost:3000.

public/index.html

public/index.html

```
<!doctype html>
<html lang="ru"><head><meta charset="utf-8"><meta name="viewport"
content="width=device-width, initial-scale=1"><title>Internet Shop</title><link
rel="stylesheet" href="/css/style.css"></head>
<body><main><h1>Internet Shop</h1><section><h2>Добавить товар</h2>
<form id="product-form"><label>Название <input id="name" required
maxlength="150"></label><label>Цена, ₸ <input id="price" type="number"
required></label><label>Категория <input id="category" required></label><label><input
id="inStock" type="checkbox" checked> В
наличии</label><button>Добавить</button></form></section>
<p id="message" role="status" aria-live="polite"></p><section><h2>Товары</h2><div
id="products"></div></section></main><script type="module"
src="/js/app.js"></script></body></html>
```

Листинг 1. public/index.html

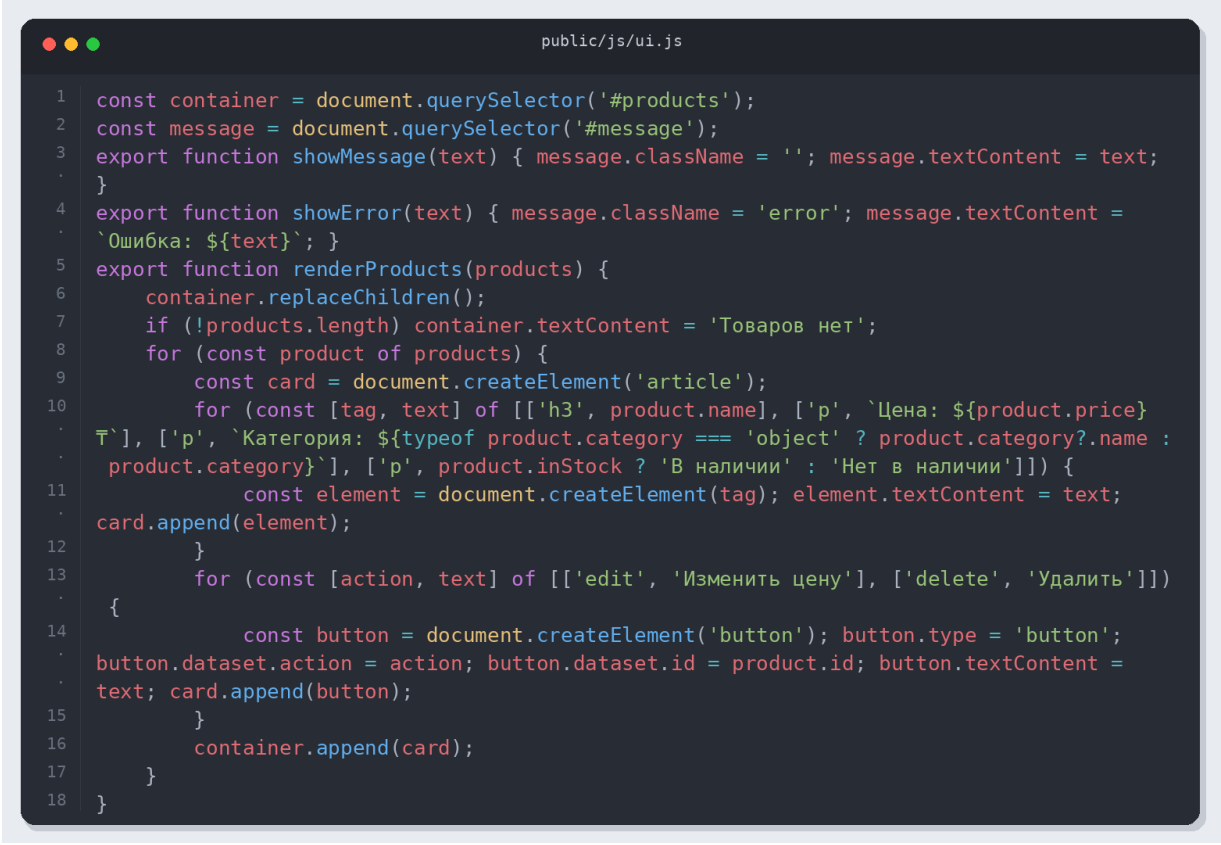
public/js/api.js

public/js/api.js

```
async function request(url, options = {}) {
    const response = await fetch(url, options);
    const data = response.status === 204 ? null : await response.json();
    if (!response.ok) throw new Error(data?.errors?.join('; ') || data?.message ||
data?.error || `HTTP ${response.status}`);
    return data;
}
export function getProducts() { return request('/api/products'); }
export function createProduct(product) { return request('/api/products', { method:
'POST', headers: { 'Content-Type': 'application/json', 'x-api-key': 'college-demo' },
body: JSON.stringify(product) }); }
export function updateProduct(id, changes) { return request(`/api/products/${id}`, {
method: 'PATCH', headers: { 'Content-Type': 'application/json' }, body:
JSON.stringify(changes) }); }
export function deleteProduct(id) { return request(`/api/products/${id}`, { method:
'DELETE' }); }
```

Листинг 2. public/js/api.js

public/js/ui.js



Листинг 3. public/js/ui.js

public/js/app.js

public/js/app.js

```
import { getProducts, createProduct, updateProduct, deleteProduct } from './api.js';
import { renderProducts, showMessage, showError } from './ui.js';
async function loadProducts() { renderProducts(await getProducts()); }
document.addEventListener('DOMContentLoaded', () => loadProducts().catch(error =>
showError(error.message)));
const form = document.querySelector('#product-form');
form.addEventListener('submit', async event => {
    event.preventDefault();
    const product = { name: form.querySelector('#name').value.trim(), price:
Number(form.querySelector('#price').value), category:
form.querySelector('#category').value.trim(), inStock:
form.querySelector('#inStock').checked };
    try { await createProduct(product); form.reset(); await loadProducts();
showMessage('Товар успешно добавлен'); }
    catch (error) { showError(error.message); }
});
document.querySelector('#products').addEventListener('click', async event => {
    const button = event.target.closest('button[data-action]');
    if (!button) return;
    const id = Number(button.dataset.id);
    try {
        if (button.dataset.action === 'delete') {
            if (!confirm('Удалить этот товар?')) return;
            await deleteProduct(id);
        } else {
            const value = prompt('Введите новую цену');
            if (value === null) return;
            if (!value.trim() || !Number.isFinite(Number(value))) throw new Error('Цена
должна быть числом');
            await updateProduct(id, { price: Number(value) });
        }
        await loadProducts(); showMessage('Изменения сохранены');
    } catch (error) { showError(error.message); }
});
```

Листинг 4. public/js/app.js

Проверка результата

Проверено: 05.11.2026 13:44 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Обязательные endpoints

• HTTP-статусы и содержимое ответов

• Полный CRUD-цикл

• Совместная фильтрация

• Уникальные id после удаления

• Chromium: форма POST, кнопки PATCH/DELETE, ошибки отрицательной цены, обновление DOM, отсутствие pageerror

HTTP-запросы

GET /about → 200

GET /api/status → 200

GET /api/shop → 200

GET /api/products → 200

GET /api/categories → 200

GET /api → 200

GET /api/orders → 200

GET /api/products/3 → 200

GET /api/products/999 → 404

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?inStock=false → 200

POST /api/products → 403

POST /api/products → 403

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 400

PATCH /api/products/1 → 400

PATCH /api/products/1 → 400

POST /api/products → 201

GET /api/products/6 → 200

PATCH /api/products/6 → 200

PATCH /api/products/6 → 400

GET /api/products/6 → 200

PATCH /api/products/4/settings?notify=true → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

PATCH /api/products/6 → 404

DELETE /api/products/6 → 404

POST /api/products → 201

GET /api/unknown → 404

POST /api/orders → 201

GET /api/orders/1 → 200

PATCH /api/orders/1 → 200

GET /api/orders/999 → 404

Примеры ответов

POST /api/products → 201


```
{
  "name": "SSD 2TB",
  "price": 65000,
  "category": "Components",
  "inStock": true,
  "id": 6
}
```

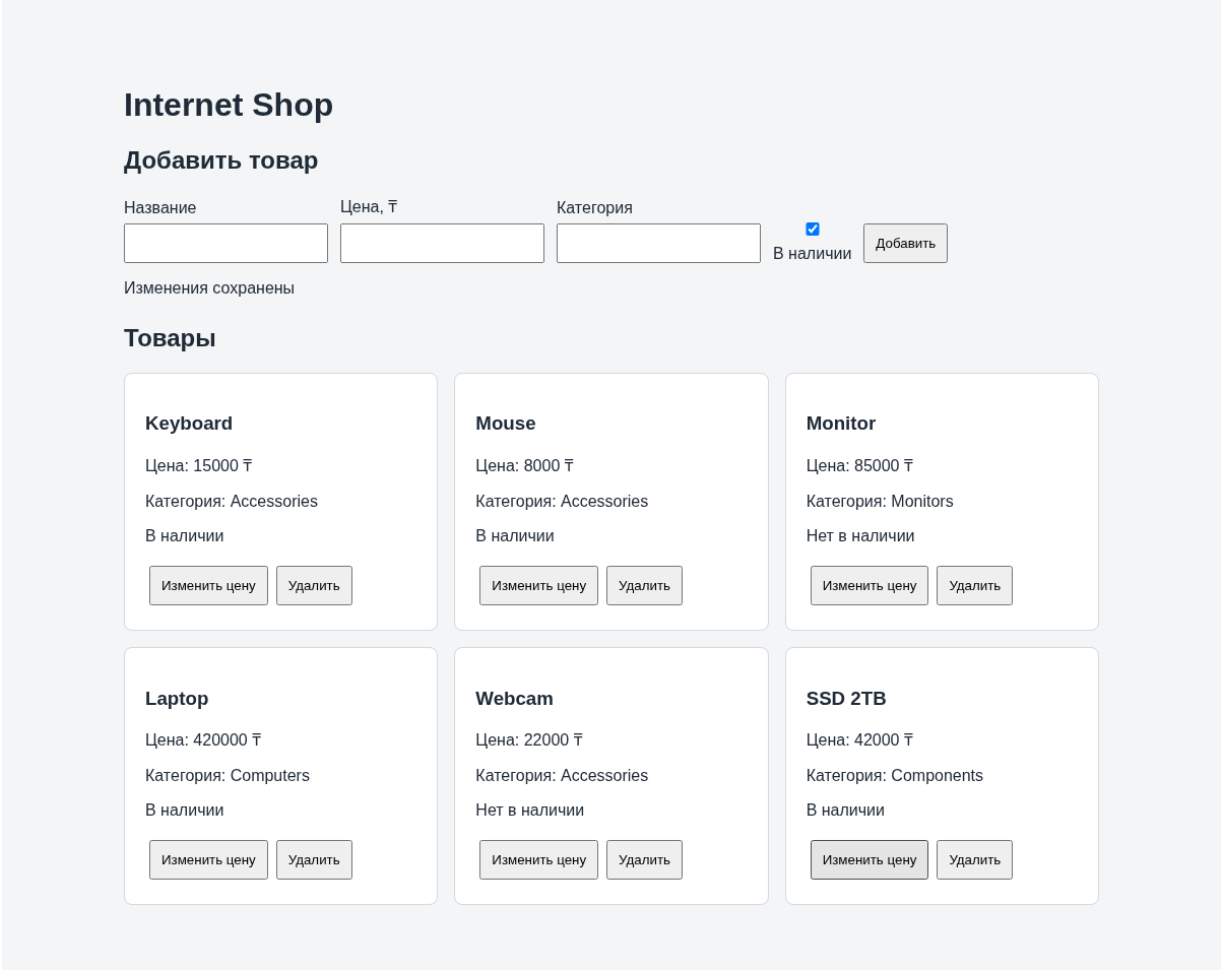

PATCH /api/products/6 → 200


```
{
  "name": "SSD 2TB",
  "price": 42000,
  "category": "Components",
  "inStock": false,
  "id": 6
}
```


DELETE /api/products/6 → 200


```
{
  "name": "SSD 2TB",
  "price": 42000,
  "category": "Components",
  "inStock": false,
  "id": 6
}
```

Проверка в браузере Chromium



Скриншот работающего приложения после выполнения действий через интерфейс.

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.